

are times when you just need to run multiple instances of Linux, for example. Then why emulate the whole PC? VT and SVM technologies virtualise the CPU very well, but the various buses and the devices sitting on them need to be emulated. This hits the performance of the virtual machines.

As an example, let us take the cases of QEMU, Xen, KVM and UML. This comparison is kind of funny, since the guys who wrote these software, never wanted to end up in a table like Table 1. This is like comparing apples to oranges, but all we want to understand from this table is whether the VMM can run an unmodified operating system, at what level it runs, and how the performance is compared to native running it.

Figures 1 and 2 explain the Xen and QEMU architecture.

Introducing OpenVZ

Let us suppose you want to run only Linux, but want to make full use of a physical server. You can run multiple instances of Linux for hosting purposes, education and in testing environments, for example. But do you have to emulate a full PC to run these virtual, multiple instances? Not really. A solution like User Mode Linux (UML) lets you run Linux on the Linux kernel, where each Linux is a separate, isolated instance.

To get a simplified view of a Linux system, let us take three crucial components that make up a system. They are: the kernel, the root filesystem, and the processes that are created as the system boots up and runs. The kernel is, of course, the core of the operating system; the root filesystem is what holds the programs and the various configuration files; and the processes are running instances of the programs created from binaries on the root file system. They are created as the system boots up and runs.

In UML, there is a host system and then there are guests. The host system has a kernel, and the root file system and its set of processes. Each guest has a kernel, a root file system and its own set of processes.

Under OpenVZ, things are a bit different. There is a single kernel and there are multiple root file systems. The guest's root file systems are directory trees under the host file system. A guest under OpenVZ is called a Virtual Environment (VE) or Virtual Private Server (VPS). Each VPS is identified using a name or a number, where VPS 0 is the host itself. Processes created by these VEs remain isolated from others. That is, if VPS 101 creates five processes and VPS 102 creates seven, they can't 'see' each other. This may sound a lot like *chroot* jails, but you must note the differences as well.

A *chroot* jail provides only filesystem isolation. The processes in a *chroot* jail still share processes, networks and other namespaces with the host. For example, if you run *ps -e* from a *chroot* jail, you still see a list of system-wide processes. If you run a socket program from the

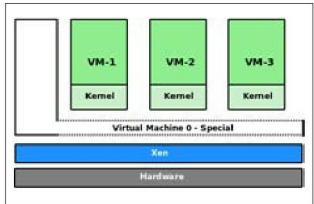


Figure 1: The Xen architecture

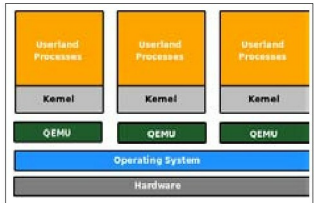


Figure 2: The QEMU architecture

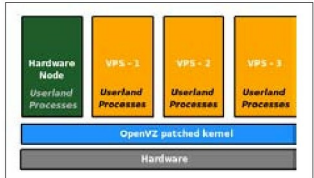


Figure 3: The OpenVZ architecture

chroot environment and listened on localhost, you can connect to it from outside the *chroot* jail. This simply means there is no isolation at the process or the network level. You can also verify this by running *netstat -a* from the *chroot* jail. You will be able to see the status of system wide networking connections.

OpenVZ is rightly called a container technology. In case of OpenVZ, there is no real virtual machine. The OpenVZ kernel is a modification of the Linux kernel that isolates namespaces and contains or separates processes created by one VPS from another (see Figure 3). By doing so, the overhead of running multiple kernels is avoided and maximum performance is obtained. In fact, the worse case overhead compared to native performance in